

Milestone 3

RAG Pipeline

IITM BS Data Science & Applications | Smart MCQ Solver Challenge | July 2026

1. Overview

This milestone is about building a Retrieval-Augmented Generation (RAG) pipeline for the Smart MCQ Solver dataset. The dataset has multiple-choice questions, each with five options and one correct answer. The pipeline retrieves relevant context from a knowledge base before passing the question to a language model for classification.

Eight questions tested each component of this pipeline in sequence: a zero-shot baseline, FAISS-based dense retrieval, cross-encoder reranking, token counting, RAG with correct and adversarial context, hit rate over the first 100 rows, and a full-pipeline MAP@3 evaluation over rows 0 to 19.

The progression is deliberate. Each question builds on the last. You cannot evaluate MAP@3 without understanding what hit rate means, and you cannot understand retrieval quality without first knowing the zero-shot baseline.

2. Knowledge Base Construction

The knowledge base (KB) stores exactly one document per training row: the text of the correct answer option. If row i has answer letter 'C', then `kb[i] = train.iloc[i]['C']`.

This is a deliberate design choice. In production RAG, you would retrieve from Wikipedia, documents, or a corpus of long passages. Here, the KB is built from ground truth. That makes retrieval success directly measurable: a hit is when the top-K retrieved docs contain the correct answer for a query.

We encoded all KB documents into 384-dimensional dense vectors using **all-MiniLM-L6-v2** from the sentence-transformers library. MiniLM is a distilled transformer model trained on large-scale sentence similarity tasks. It is fast enough to encode thousands of sentences in seconds and accurate enough for a strong retrieval baseline.

When we looked into model options, we found that MiniLM consistently appears in retrieval benchmarks on the BEIR (Benchmarking IR) suite as a strong performer in the sub-100M parameter category. Its 384-dimensional output also keeps the FAISS index lightweight.

3. Dense Retrieval with FAISS

FAISS (Facebook AI Similarity Search) is a library for nearest-neighbor search over dense vectors. We used **IndexFlatL2**, which computes exact Euclidean (L2) distances between the query vector and every vector in the index.

Exact search is fine for a KB of this size. For millions of documents, you would use an approximate index like IndexIVFFlat or IndexHNSW, which sacrifice a small amount of recall to gain orders-of-magnitude speedup. We looked at the FAISS documentation and the ANN-benchmarks dataset to understand these tradeoffs before picking IndexFlatL2 for this prototype.

Q2 tested whether FAISS, given row 150's prompt as a query, could retrieve kb[150] in its top-10 results. This is a relevance test: does the question embed close to the answer in MiniLM's vector space? That is not guaranteed, because questions and answers often use very different vocabulary.

The embedding pipeline here is simple: encode the query prompt with the same MiniLM model used for the KB, then call `index.search(query_vector, 10)`. We also had to make sure the query encoding and KB encoding use the same normalization settings, otherwise the distance calculations would be skewed.

4. Cross-Encoder Reranking

Bi-encoders like MiniLM process the query and each document independently. That is what makes them fast: you precompute KB embeddings once, then find nearest neighbors with a single matrix operation at inference time.

The limitation is that there is no direct interaction between query and document inside the encoder. Cross-encoders fix this by processing a (query, document) pair jointly through the transformer. The model can attend across both texts, which gives richer relevance signals.

We used **cross-encoder/ms-marco-MiniLM-L-6-v2**, trained on the MS-MARCO passage reranking dataset. MS-MARCO contains real Bing search queries paired with human-judged relevant passages, making it a good proxy for our question-to-answer reranking task.

The standard two-stage setup: FAISS retrieves the top-10 candidates cheaply, then the cross-encoder scores all 10 (query, document) pairs and reranks them. **Q3** asked where kb[150] lands after reranking. The cross-encoder sometimes improves the rank of the true document and sometimes does not, depending on how semantically distinct the true answer is from the other nine candidates.

Cross-encoder inference is slow because you cannot precompute scores. Every query requires a separate forward pass for each candidate. For Q8's full pipeline over 20 rows, this was the main bottleneck.

5. Zero-Shot Classification with BART

We used **facebook/bart-large-mnli** for classification. BART is a sequence-to-sequence model, and the MNLI fine-tuned variant is trained on the Multi-Genre Natural Language Inference corpus. The zero-shot trick: treat the question (or RAG string) as a premise and each answer option as a hypothesis. The model scores the probability of entailment for each option, and the option with the highest entailment score becomes the predicted answer.

The HuggingFace zero-shot-classification pipeline automates this. You pass the text and a list of candidate labels, and get back scores summing to 1. **Q1** established the no-retrieval baseline: what probability does the model assign to the correct answer for row 150 when given only the prompt and five options?

Q5 and Q6 then introduced context. Q5 prepended kb[150] (the true correct document) to the question. Q6 prepended kb[999], a completely unrelated document. The question was: does the right context help, and does the wrong context hurt?

This pair of experiments is the most conceptually important in the milestone. If adversarial context (Q6) drops the probability of the correct answer below the no-context baseline (Q1), then retrieval quality becomes a hard gate on overall system performance. A RAG system that retrieves garbage will actively make the model worse.

6. Token Counting and Context Length

Q4 asked for the BERT token count of the RAG string built for row 42: the five retrieved documents concatenated with spaces, then formatted as:

```
Context: [doc1] [doc2] [doc3] [doc4] [doc5] Question: [prompt]
```

We used **bert-base-uncased** from the HuggingFace transformers library. BERT has a hard context window of 512 tokens. BART-large, used for classification, can handle 1024 tokens. If a RAG string exceeds these limits, the model simply truncates the input, and the beginning of the context may be cut off.

We counted tokens both with and without special tokens ([CLS] and [SEP]) because different grading rubrics count differently. The token count depends entirely on the length of the five retrieved documents. A KB with long answer texts will produce RAG strings that overflow standard context windows.

In practice, this is a real engineering constraint. A common solution is to truncate each retrieved document to a fixed number of tokens before concatenation, or to retrieve fewer documents. Another option is to use models with longer context windows.

7. Evaluation: Hit Rate and MAP@3

Hit Rate (Q7)

Hit rate over the first 100 rows: for each row, retrieve the top-5 documents with FAISS. A hit is when the correct answer text for that row appears in (or as) any of the five retrieved

documents. The hit rate is the fraction of rows where at least one of the five retrieved docs contains the correct answer.

This metric tells you whether the retrieval component is doing useful work. A hit rate of 60% means the correct answer is in the top-5 retrieved docs 60% of the time. That is the ceiling for how often RAG can help: if the correct answer is not retrieved, the LLM cannot benefit from seeing it.

We implemented the hit check with a string containment condition rather than exact equality, because retrieved documents might be truncated or slightly different from the stored answer text.

Full Pipeline MAP@3 (Q8)

MAP@3 (Mean Average Precision at rank 3) measures whether the correct answer appears in the model's top-3 predictions, and where. The formula per row is:

```
AP@3 = 1 / rank_of_correct_answer (if correct answer in top-3, else 0)
```

MAP@3 is the mean of AP@3 across all evaluated rows. A perfect score of 1.0 means the correct answer was ranked first for every row. A score of 0.5 means correct answers appeared, on average, at rank 2.

The Q8 pipeline for each row: (1) encode the prompt, (2) retrieve top-5 with FAISS, (3) rerank with the cross-encoder, (4) take the best-scored doc, (5) build a RAG string, (6) run zero-shot classification with BART, (7) map the top-3 scored labels back to answer letters A through E, (8) compute AP@3 against the ground truth answer.

We ran this over 20 rows. Each row requires two model inferences: one cross-encoder call (scoring 5 pairs) and one BART classification call. Both are slow on CPU. On a Kaggle GPU, the full 20-row evaluation still takes several minutes.

8. Key Findings and Observations

The adversarial context result is the most telling.

Comparing Q1 (no context), Q5 (true context), and Q6 (wrong context) in sequence is the cleanest test of whether RAG is worth the engineering complexity. If Q6 scores lower than Q1, retrieval quality is not just helpful: it is necessary. A retrieval failure actively hurts the system, not just fails to help it.

Two-stage retrieval has a clear rationale.

The bi-encoder + cross-encoder combination is not arbitrary. Bi-encoders are fast because they decouple query and document encoding. Cross-encoders are more accurate because they model query-document interactions. Using a cross-encoder on all KB documents would be too slow. Using only a bi-encoder loses relevance signal. Two stages give you the best of both.

Context window limits are a real constraint, not a footnote.

The token count exercise (Q4) is directly relevant to what happens when you run BART on a long RAG string. BART-large's 1024-token limit can be exceeded by five medium-length documents. In any real deployment, you would need explicit truncation or chunking logic. This is why retrieval system design and model selection are not independent choices.

The knowledge base design limits generalization.

Building the KB from ground truth correct answers means the system needs access to training labels at inference time. That is a strong assumption. A more realistic RAG setup would retrieve from an external corpus (Wikipedia, textbooks) rather than from the dataset's own labels. The evaluation is clean because hits are measurable, but the approach does not transfer directly to production without a labeled KB.

MiniLM and MS-MARCO are a consistent pairing.

Both the bi-encoder (all-MiniLM-L6-v2) and the cross-encoder (ms-marco-MiniLM-L-6-v2) are trained on or with MS-MARCO data. This is intentional in the sentence-transformers library: the two models are designed to work together in a two-stage retrieval system. Mixing a BEIR-trained bi-encoder with an MS-MARCO cross-encoder can introduce distributional mismatch. Keeping both from the same training lineage is good practice.

9. Libraries and Tools Used

The entire pipeline runs in Python on a Kaggle GPU notebook. Key dependencies:

sentence-transformers

Bi-encoder and cross-encoder inference. Wraps HuggingFace models with a clean API for encoding and similarity scoring.

faiss-cpu

Vector similarity search. IndexFlatL2 for exact nearest-neighbor search over the KB embeddings.

transformers (HuggingFace)

BART-large-MNLI zero-shot classification pipeline and BERT tokenizer for token counting.

pandas / numpy

Data loading, row access, and numeric operations across the pipeline.